End-to-End Order Fulfilment & Customer Experience Reporting

A single order-level view from Shopify purchase to delivery, support outcome and factory accountability.

Objective

Create one reliable chain of evidence for every order: when it arrived, what the factory did, what the carrier did, whether it reached the customer on time and in full, and whether the customer subsequently contacted support.

1	2	3	4	5	6
SHOPIFY Order created	**FACTORY** Received / picked	**CARRIER** Collected	**CUSTOMER** Delivered	**COMMSLAYER** Contact / issue	**REPORTING** OTIF + scorecard

1. Business problem

Today, the operational chain is fragmented. Shopify shows the customer order, the factory and carrier hold fulfilment events, and Commslayer holds the customer complaint history. Without joining those data sets at order level, it is difficult to distinguish a factory delay, a carrier delay, a packing error or a customer-side issue.

- No single timeline for each order from purchase to final delivery.
- Limited ability to calculate credible On-Time-In-Full (OTIF) performance.
- Customer complaints are visible as support workload, but not consistently connected back to operational failures.
- Factory performance discussions rely on anecdotes rather than measurable evidence, trends and cost of failure.

2. Target outcome

Create an order-level data model in which the Shopify order ID is the common key. Every operational and customer-service event is attached to that order, producing a complete lifecycle and a defensible factory scorecard.

OTIF % On time AND in full	**Dispatch SLA** Order to carrier handover	**Transit time** Carrier collection to delivery
Issue rate Orders generating CS contact	**Failure mix** Late / missing / wrong / damaged	**Cost of failure** Refunds + replacements + support

3. Proposed order lifecycle

Stage	Core timestamp / field	Primary source	What it proves
Order placed	order_created_at	Shopify	Demand entered the system
Factory received	factory_received_at	Factory / WMS	Factory had the order
Picked / packed	picked_at / packed_at	Factory / WMS	Warehouse processing time
Carrier collected	carrier_collected_at	Carrier	Physical handover occurred
Delivered	delivered_at	Carrier	Customer received parcel
Contents	ordered qty vs shipped / missing qty	Shopify + factory + CS	Whether order was in full
Support contact	first_contact_at + channel	Commslayer	Whether fulfilment created support demand
Issue / resolution	reason + outcome + refund/replacement	Commslayer / Shopify	Failure type and business impact

4. Data architecture

Recommended MVP: use Airtable as the reporting data layer, with n8n (or a similar automation layer) to ingest and normalise events. Commslayer remains the customer-service system rather than becoming the system of record for fulfilment.

SOURCE SYSTEMS	AUTOMATION / MATCHING	REPORTING LAYER
Shopify Factory / WMS Carrier Commslayer	Webhooks / APIs / CSV Match by order ID Standardise timestamps	Airtable Orders Order Items Customer Issues
Events stay auditable	**One lifecycle per order**	**Dashboards + factory scorecard**

5. Customer-service classification

For reporting to work, support contacts need structured issue labels rather than only free-text notes. Classification can be agent-selected, AI-assisted or fully automated where confidence is high.

Suggested initial taxonomy: late delivery, not dispatched, lost in transit, missing item, wrong item, damaged/leaking, address/customer error, cancellation, other.

6. Factory scorecard

The reporting output should separate factory-controlled performance from carrier-controlled performance. This makes the data actionable and avoids disputes over where the failure occurred.

Factory performance	Carrier performance	Customer impact
Order-to-pick time	Collection-to-delivery time	% orders with support contact
On-time dispatch %	On-time delivery %	Missing / damaged / wrong-item rate
Pick/pack accuracy	Lost / exception rate	Refund and replacement value
Orders sitting before collection	Carrier SLA by service	Estimated CS handling cost

7. Example management output

Instead of: "We are getting a lot of complaints."

Report: "8.1% of orders missed dispatch SLA; 2.7% arrived with missing items; 5.4% generated a support contact. These failures drove £X of refunds/replacements and Y hours of support handling."

8. Phased implementation

Phase 1	Phase 2	Phase 3	Phase 4
Map data	**Build MVP**	**Add CS**	**Automate reporting**
Confirm every source, ID and timestamp; agree SLA definitions.	Join Shopify + carrier + factory data; create order timeline.	Connect Commslayer; classify issues and resolutions by order.	Weekly/monthly OTIF dashboard and factory scorecard.

9. Information needed before build

- What system does the factory use to receive, pick and pack orders, and can it expose timestamps through an API, webhook, export or daily CSV?
- Which carrier(s) are used and what tracking / event API is available?
- What is the agreed dispatch SLA and delivery SLA by market / service?
- Can Commslayer conversations reliably expose Shopify order number, customer email or another matching key?
- Which failure costs should be included: refunds, replacements, reshipping, discounts and estimated support time?

End state: a traceable order-level dataset that turns fulfilment and customer-service noise into measurable OTIF, root-cause and cost-of-failure reporting.